Frank Ziegler – grat Project Planning Document

– THE TEAM –

1. Project Team

Team Member	Role	Responsibilities
Frank Ziegler	CEO/CTO	1. Full-stack engineering 2. Project design and documentation 3. Deployment

– THE IDEA –

2. Project Information and Scope

What is it?

grat is a mobile app that helps tip-based employees track their income and important details about every shift they work.

What does it do?

To use the app, servers/bartenders/baristas/aesthetics professionals/etc. "add a new shift," where they can input important information about a shift they worked at their place of employment, including the total amount of money they were tipped, their sales (and sales breakdowns), the amount of hours they worked, the amount they tipped out.

The app will also specialize in trends and analytics. The app tracks stats important to the user over time and displays the data in meaningful ways. A particular example of this could be: a server at a new restaurant uses the app for 8 months after getting hired. They may like to see their average tip percentage per table over the course of the 8 months where they gained more menu knowledge, more upselling techniques, etc.

More than anything, however, it's a convenient way to track income. Oftentimes, tip-based employees have *extremely* variable income earned and disbursed at uncommon intervals. To track it over time and analyze it and average it can be incredibly insightful in understanding trends (with respect to external variables like days of the week, seasons of the year, etc.) and useful in gaining comfortability with such variability and volatility.

Users?

The targeted user group is tipped-out employees. This includes servers, bartenders, tattoo artists, beauty care professionals, haircare professionals, baristas, and more.

Backend? Data?

Data

The most important entity, aside from the users, is the `shift` entity. There are attributes of a given shift worked that are necessary, i.e. `clock_in`, `clock_out`, `uid`, `date`, etc. There are also attributes of a given shift that are not necessary, but may be useful to track. **grat** empowers users to track *their own metrics*. For example, as a server, there are shift metrics important to me that may not be important to others (like a hair stylist), such as `table_count`, `gross_sales`, `alcohol_sales`, `cocktails_sold`, etc. These are attributes only relevant to the *job that I do, personally.* This, I believe is going to be a core feature of the application and will allow me to cast a net wide enough to include multiple industries.

I am opting to use MongoDB.

NoSQL model:

```
// A shift entry:
{
    "user": {
        "uid": 1,
        "username": "frankzigs",
        "email": "frankzigs98@gmail.com"
    },
    "date": "2025-09-14",
    "in": "16:00",
    "out": "22:00",
    "metrics": {
        "tips": 250.00,
        "table_count": 8,
        "sales": 1146.28,
        "coworkers": ["lisa", "john", "jessie", "cecilia", "matt"]
    }
}
```

Backend

I am looking to settle on a data storage method. Once I do, I'll likely do research over the next week to determine the most compatible backend framework to interact with the data. I lean towards Spring Boot or FastAPI.

In any case, the API will likely follow REST structure and will have dedicated endpoints for handling interactions with the data store. The bulk of the work will be in adding new shifts and querying data for charting and tracking metrics over time. Some endpoints may be:

- `POST /users/{uid}/shifts/{shift_details}`
- `GET /users/{uid}/shift/{id}`
- `GET /users/{uid}/shifts/{date}`

- `GET /users{uid}/shifts/{sid}/metrics/{mid}`
- or whatever.

To me, modularity is important. I'd probably design my backend so that it retains separation of services, i.e.:

```
gratBackend/
|- shifts/
|- metrics/
|- reporting/
|- user/
```

Deliverables

1. React Native UI Pages and Components
   - Authentication/Onboarding
   - Main container for
     - Dashboard
     - Calendar
     - Reporting
     - Settings
   - Navigation
   - Charts/Reporting
   - User settings
2. Java Spring backend
3. MongoDB database
4. Cloud hosting for backend
5. Deployment to app stores?

3. Project Risks, Constraints, and Feasibility

Feasibility

The project is definitely feasible. I don't personally find it to be that complex of an application. I am looking to spiffy it up by building a base application and porting my server-side code to a microservices architecture and learn a little bit about locally caching data with something like Redis to learn more industry-level technologies but aside from that, it's not very complicated. It's a React Native frontend with a REST-compliant backend and MongoDB.

Project Limitations

1. This project can absolutely be completed on time (although, deployment to app stores could be a lofty goal).
2. This project could integrate with other systems depending on what they are. A particular use case may be an income distribution application/software bundle for places of employment looking to use a third party app to manage all pay distributions. An example of this may be TipHaus, which is exactly that. Their reporting isn't amazing and neither is their distribution service as it's often buggy.
3. The only limitation to the success of the project is my UI skills. I am not that intuitive of a UI designer. I also am not *a master* of React and often find myself struggling with basic syntax. I also think being able to chart things well is going to be an important feature of the application, and I'm not aware of any decent charting libraries in React Native, so they may have to be hand-drawn with Skia. The other limitation may be my unfamiliarity with NoSQL databases but so far it's not that much of a bottleneck.
4. I rank technical risk kind of high. I'm wondering if the authentication mechanism for my application is thoughtful considering a lot of mobile apps nowadays use local storage for their data. I'm choosing session-persistence, but I wonder if that may change in the future. Users owning their data may prove to be paramount to session persistence. Otherwise, the tech stack I am using is tried and true. This also leads to security risk being ranked high as well if I were to expand the application. We are tracking people's income, after all.
5. No technical unknowns that I can foresee.
6. No single point of failures either. My data is simple. My UI is forecasted to be clean and intuitive. My server-side logic uses REST.

Risk Assessment

Project Management Risks

There are no big project management risks to mention unless I were to add additional functionality to this application. A simple tip-tracker with customizable metrics should not be more than a 2-3 month project.

I do have plans to expand grat to be only part of a larger application but that will not occur for a long time, if ever. I mostly chose this project because I need my own personal tip tracker.

Technical Risks

I do have a couple technical risks to note:

1. Manual user authentication is not easy across HTTP with Spring. It is a pain with CORS. It took me days to figure out in Software Engineering.
2. Charting in React Native is going to be a little tricky if I want to nail it. I don't know of nor can I find any decent charting libraries.

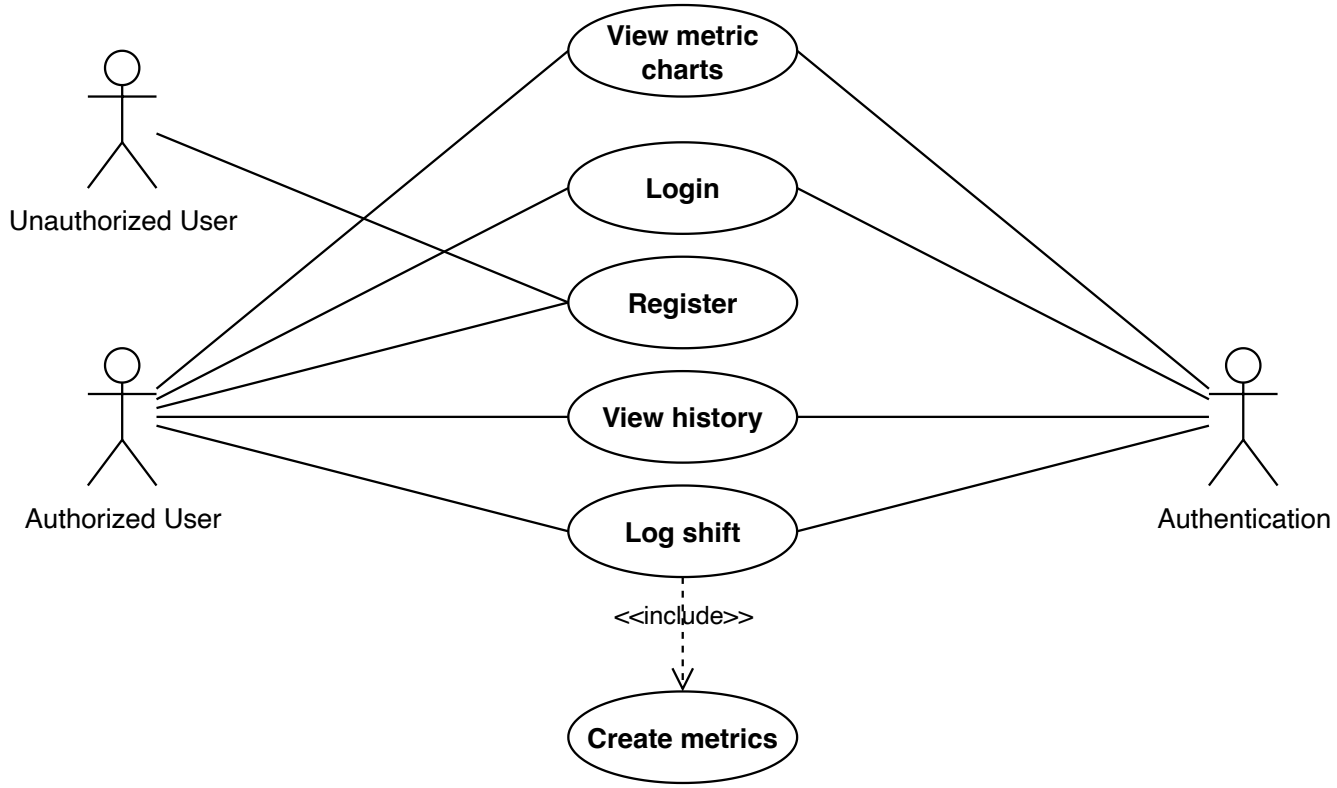
Requirement Risks

One ambiguous requirement I have is the charting feature of the application. How exactly do I display the custom metrics given by users in a way that's meaningful? We have numeric and categorical data. So,

line charts? Bar charts? Why?

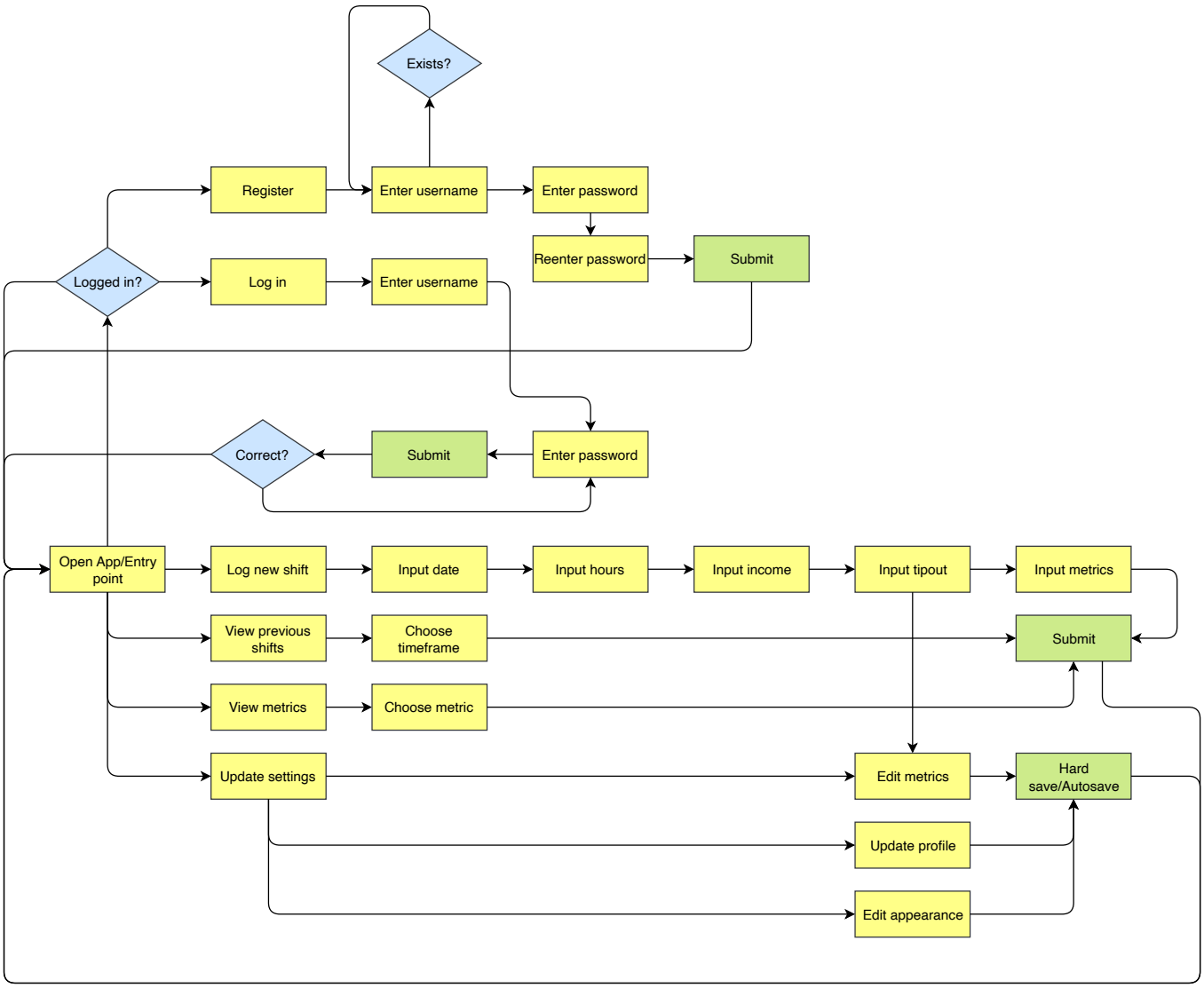
– DESIGN –

4. Map the Project Architecture: Use Case Diagram



gratUseCase.svg

Visualize the Product: Map the Project Architecture [Flow Chart or Activity Diagram]



gratArchFlowchart.svg

– DESIGN METHODOLOGY –

6. Project Methodology

To track and document progress, changes, and setbacks, I am a firm believer in detailed note-taking and logging. The log for this project can be accessed here: grat Log.

- Every session I have with building this application will get a log review here from here on out. I will treat it like a journal.

The only other tracking tools I plan to use are Todoist for managing the backlog and Git for managing version controlling and tracking updates.

7. Project Language and Tools: Architecture Requirements

The development and production environments for now:

- Frontend: local instance on phone testing through ExpoGo app

- Backend: localhost until I am ready to deploy
- Database: localhost

The product requirements:

1. Inputs and outputs: user input, server-retrieved data displayed via charts and logs
2. Local spring server hosting a model-controller-repository-service structure
3. AWS or free alternative host
4. Software requirements
    1. Spring Boot
    2. Spring Security
    3. Spring Web
    4. RN Skia
    5. Expo
5. Database requirements: single doc or two-doc database for users and shifts
6. Non-functional requirements: an Android to test with (as I only have an iPhone)